

Doubly Linked List

1.Insertion at Beginning.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void insert()
14 {
15     struct node *temp = head;
16
17     while(temp != NULL)
18     {
19         printf("%d <-> ", temp->data);
20         temp = temp->next;
21     }
22     printf("NULL\n");
23 }
24
25 void insertatbegin(int value)
26 {
27     struct node *newnode;
28
29     newnode = (struct node*)malloc(sizeof(struct node));
30     newnode->data = value;
31     newnode->prev = NULL;
32     newnode->next = head;
33
34     if(head != NULL)
35     {
36         head->prev = newnode;
37     }
38
39     head = newnode;
40 }
41
42 int main()
43 {
44     insertatbegin(201);
45     insertatbegin(301);
46
47     printf("Before Insertion:\n");
48     insert();
49
50     insertatbegin(101);
51
52     printf("After Insertion at Beginning:\n");
53     insert();
54     return 0;
55 }
```

Before Insertion:
301 <-> 201 <-> NULL
After Insertion at Beginning:
101 <-> 301 <-> 201 <-> NULL

=== Code Execution Successful ===

Before Insertion:
301 <-> 201 <-> NULL
After Insertion at Beginning:
101 <-> 301 <-> 201 <-> NULL

=== Code Execution Successful ===

2.Insertion at End.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void insert()
14 {
15     struct node *temp = head;
16
17     while(temp != NULL)
18     {
19         printf("%d <-> ", temp->data);
20         temp = temp->next;
21     }
22     printf("NULL\n");
23 }
24
25 void insertatend(int value)
26 {
27     struct node *newnode, *temp;
28
29     newnode = (struct node*)malloc(sizeof(struct node));
30     newnode->data = value;
31     newnode->next = NULL;
32
33     if(head == NULL)
34     {
35         newnode->prev = NULL;
36         head = newnode;
37     }
38     else
39     {
40         temp = head;
41         while(temp->next != NULL)
42         {
43             temp = temp->next;
44         }
45
46         temp->next = newnode;
47         newnode->prev = temp;
48     }
49 }
50
51 int main()
52 {
53     insertatend(101);
54     insertatend(201);
```

Before Insertion:
101 <-> 201 <-> NULL
After Insertion at End:
101 <-> 201 <-> 301 <-> NULL
=== Code Execution Successful

Before Insertion:
101 <-> 201 <-> NULL
After Insertion at End:
101 <-> 201 <-> 301 <-> NULL
=== Code Execution Successful

```

45     temp->next = newnode;
46     newnode->prev = temp;
47 }
48 }
49 }
50
51 int main()
52 {
53     insertatend(101);
54     insertatend(201);
55
56     printf("Before Insertion:\n");
57     insert();
58
59     insertatend(301);
60
61     printf("After Insertion at End:\n");
62     insert();
63     return 0;
64 }

```

Before Insertion:
101 <-> 201 <-> NULL
After Insertion at End:
101 <-> 201 <-> 301 <-> NULL

=== Code Execution Successful

3. Insertion at Position.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void insert()
14 {
15     struct node *temp = head;
16
17     while(temp != NULL)
18     {
19         printf("%d <-> ", temp->data);
20         temp = temp->next;
21     }
22     printf("NULL\n");
23 }
24
25 void insertatposition(int value, int pos)
26 {
27     struct node *newnode, *temp;
```

```
Before Insertion:
101 <-> 201 <-> 401 <-> NULL
After Insertion at Position 3:
101 <-> 201 <-> 301 <-> 401 <-> NULL
```

=== Code Execution Successful ===

```
28     int i;
29
30     newnode = (struct node*)malloc(sizeof(struct node));
31     newnode->data = value;
32
33     if(pos == 1)
34     {
35         newnode->prev = NULL;
36         newnode->next = head;
37         if(head != NULL)
38             head->prev = newnode;
39         head = newnode;
40         return;
41     }
42
43     temp = head;
44     for(i = 1; i < pos-1; i++)
45     {
46         temp = temp->next;
47     }
48
49     newnode->next = temp->next;
50     newnode->prev = temp;
51
52     if(temp->next != NULL)
53         temp->next->prev = newnode;
54
```

```
Before Insertion:
101 <-> 201 <-> 401 <-> NULL
After Insertion at Position 3:
101 <-> 201 <-> 301 <-> 401 <-> NULL
```

=== Code Execution Successful ===

```

54
55     temp->next = newnode;
56 }
57
58 int main()
59 {
60     insertatposition(101,1);
61     insertatposition(201,2);
62     insertatposition(401,3);
63
64     printf("Before Insertion:\n");
65     insert();
66
67     insertatposition(301,3);
68
69     printf("After Insertion at Position 3:\n");
70     insert();
71
72     return 0;
73 }

```

```

Before Insertion:
101 <-> 201 <-> 401 <-> NULL
After Insertion at Position 3:
101 <-> 201 <-> 301 <-> 401 <-> NULL

```

```

=== Code Execution Successful ===

```

4. Deletion at Beginning.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void delete()
14 {
15     struct node *temp = head;
16
17     while(temp != NULL)
18     {
19         printf("%d <-> ", temp->data);
20         temp = temp->next;
21     }
22     printf("NULL\n");
23 }
24
25 void deleteatbegin()
26 {
27     struct node *temp;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at Beginning:
201 <-> 301 <-> NULL
```

=== Code Execution Successful

```
29     if(head == NULL)
30     {
31         printf("List is empty\n");
32         return;
33     }
34
35     temp = head;
36     head = head->next;
37
38     if(head != NULL)
39         head->prev = NULL;
40
41     free(temp);
42 }
43
44 int main()
45 {
46     struct node *n1,*n2,*n3;
47
48     n1 = (struct node*)malloc(sizeof(struct node));
49     n2 = (struct node*)malloc(sizeof(struct node));
50     n3 = (struct node*)malloc(sizeof(struct node));
51
52     n1->data = 101; n1->prev = NULL; n1->next = n2;
53     n2->data = 201; n2->prev = n1; n2->next = n3;
54     n3->data = 301; n3->prev = n2; n3->next = NULL;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at Beginning:
201 <-> 301 <-> NULL
```

=== Code Execution Successful

```

48     n1 = (struct node*)malloc(sizeof(struct node));
49     n2 = (struct node*)malloc(sizeof(struct node));
50     n3 = (struct node*)malloc(sizeof(struct node));
51
52     n1->data = 101; n1->prev = NULL; n1->next = n2;
53     n2->data = 201; n2->prev = n1; n2->next = n3;
54     n3->data = 301; n3->prev = n2; n3->next = NULL;
55
56     head = n1;
57
58     printf("Before Deletion:\n");
59     delete();
60
61     deleteatbegin();
62
63     printf("After Deletion at Beginning:\n");
64     delete();
65
66     return 0;
67 }

```

Before Deletion:

101 <-> 201 <-> 301 <-> NULL

After Deletion at Beginning:

201 <-> 301 <-> NULL

=== Code Execution Successful ===

5. Deletion at End.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void delete()
14 {
15     struct node *temp = head;
16
17     while(temp != NULL)
18     {
19         printf("%d <-> ", temp->data);
20         temp = temp->next;
21     }
22     printf("NULL\n");
23 }
24
25 void deleteatend()
26 {
27     struct node *temp;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at End:
101 <-> 201 <-> NULL
```

=== Code Execution Successful ===

```
29     if(head == NULL)
30     {
31         printf("List is empty\n");
32         return;
33     }
34
35     temp = head;
36
37     if(temp->next == NULL)
38     {
39         free(temp);
40         head = NULL;
41         return;
42     }
43     while(temp->next != NULL)
44     {
45         temp = temp->next;
46     }
47
48     temp->prev->next = NULL;
49     free(temp);
50 }
51
52 int main()
53 {
54     struct node *n1,*n2,*n3;
55
56     n1 = (struct node*)malloc(sizeof(struct node));
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at End:
101 <-> 201 <-> NULL
```

=== Code Execution Successful ===


```
56     n1 = (struct node*)malloc(sizeof(struct node));
57     n2 = (struct node*)malloc(sizeof(struct node));
58     n3 = (struct node*)malloc(sizeof(struct node));
59
60     n1->data = 101; n1->prev = NULL; n1->next = n2;
61     n2->data = 201; n2->prev = n1; n2->next = n3;
62     n3->data = 301; n3->prev = n2; n3->next = NULL;
63
64     head = n1;
65
66     printf("Before Deletion:\n");
67     delete();
68
69     deleteatend();
70
71     printf("After Deletion at End:\n");
72     delete();
73
74     return 0;
75 }
```

Before Deletion:
101 <-> 201 <-> 301 <-> NULL
After Deletion at End:
101 <-> 201 <-> NULL

=== Code Execution Successful

6. Deletion at Position.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *prev;
8      struct node *next;
9  };
10
11 struct node *head = NULL;
12
13 void delete()
14 {
15     struct node *temp = head;
16
17     while(temp != NULL)
18     {
19         printf("%d <-> ", temp->data);
20         temp = temp->next;
21     }
22     printf("NULL\n");
23 }
24
25 void deleteatposition(int pos)
26 {
27     struct node *temp;
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> 401 <-> NULL
After Deletion at Position 3:
101 <-> 201 <-> 401 <-> NULL
```

=== Code Execution Successful ===

```
28     int i;
29
30     if(head == NULL)
31     {
32         printf("List is empty\n");
33         return;
34     }
35
36     temp = head;
37
38     if(pos == 1)
39     {
40         head = temp->next;
41         if(head != NULL)
42             head->prev = NULL;
43         free(temp);
44         return;
45     }
46
47     for(i = 1; i < pos; i++)
48     {
49         temp = temp->next;
50     }
51
52     if(temp->next != NULL)
53         temp->next->prev = temp->prev;
54
```

```
Before Deletion:
101 <-> 201 <-> 301 <-> 401 <-> NULL
After Deletion at Position 3:
101 <-> 201 <-> 401 <-> NULL
```

=== Code Execution Successful ===

```

55     if(temp->prev == NULL)
56     {
57         temp->prev->next = temp->next;
58     }
59     free(temp);
60 }
61 int main()
62 {
63     struct node *n1,*n2,*n3,*n4;
64
65     n1 = (struct node*)malloc(sizeof(struct node));
66     n2 = (struct node*)malloc(sizeof(struct node));
67     n3 = (struct node*)malloc(sizeof(struct node));
68     n4 = (struct node*)malloc(sizeof(struct node));
69
70     n1->data = 101; n1->prev = NULL; n1->next = n2;
71     n2->data = 201; n2->prev = n1; n2->next = n3;
72     n3->data = 301; n3->prev = n2; n3->next = n4;
73     n4->data = 401; n4->prev = n3; n4->next = NULL;
74
75     head = n1;
76
77     printf("Before Deletion:\n");
78     delete();
79
80     deleteatposition(3);
81
82     printf("After Deletion at Position 3:\n");
83     delete();
84
85     return 0;
86 }

```

Before Deletion:
101 <-> 201 <-> 301 <-> 401 <-> NULL
After Deletion at Position 3:
101 <-> 201 <-> 401 <-> NULL

=== Code Execution Successful ===